

Introduction to Python

- High-level, interpreted programming language
- Developed by **Guido van Rossum** in 1991
- Emphasizes **readability** and **simplicity**
- Supports multiple programming paradigms:
 - Object-oriented
 - Procedural
 - Functional

Why Python?

- Easy to learn and use
- Vast standard libraries and community support
- Platform-independent
- Suitable for:
 - Web Development
 - Data Science

Cont...

- Machine Learning
- Automation
- Scripting
- Desktop Applications

Python Versions

- Python 2.x (Legacy)
- Python 3.x (Current standard)
- Key Differences:
 - Print statement vs. `print()` function
 - Integer division behavior
 - Unicode support

Python Features

- Interpreted and interactive
 - Dynamically typed
 - Garbage-collected
 - Extensive standard library
 - Easy integration with C/C++/Java
 - Embeddable and extensible
-

Python Syntax Basics

- Variables and Data Types
- Indentation (No braces)
- Input/Output functions
- Comments (#)
- Example:

python

```
name = input("Enter your name: ")  
print(f"Hello, {name}!")
```

Data Types and Structures

- Basic: `int`, `float`, `str`, `bool`
- Data Structures:
 - `list`, `tuple`, `set`, `dict`
- Example:

python

```
students = {"Alice": 85, "Bob": 92}
```

Control Structures

- Conditional: `if`, `elif`, `else`
- Loops: `for`, `while`
- Loop control: `break`, `continue`, `pass`
- Example:

python

```
for i in range(5):  
    print(i)
```


Functions and Modules

- Defined using `def`
- Supports default and keyword arguments
- Modules: Built-in and user-defined

python

```
def greet(name):  
    return f"Hello, {name}"
```

Object-Oriented Programming in Python

- Classes and Objects
- Inheritance and Polymorphism
- Encapsulation
- Example:

python

```
class Student:  
    def __init__(self, name):  
        self.name = name
```

Advanced Topics

- File Handling
- Exception Handling
- Generators and Iterators
- Decorators
- Lambda functions
- Comprehensions

Python Libraries and Frameworks

- Web: Django, Flask
 - Data Science: NumPy, Pandas, Matplotlib, SciPy
 - ML/AI: scikit-learn, TensorFlow, PyTorch
 - Automation: Selenium, OpenCV
-

Real-World Applications

- YouTube, Instagram, Dropbox use Python
- Data Analysis and Visualization
- Chatbots and AI systems
- Game Development
- IoT and Robotics

Python Development Tools

- IDEs: PyCharm, VS Code, Jupyter Notebook, Thonny
 - Version Control: Git & GitHub
 - Package Management: pip, conda
-

Conclusion

- Python is versatile and beginner-friendly
- Powerful for both academic and industrial applications
- Community-driven development ensures continuous evolution